



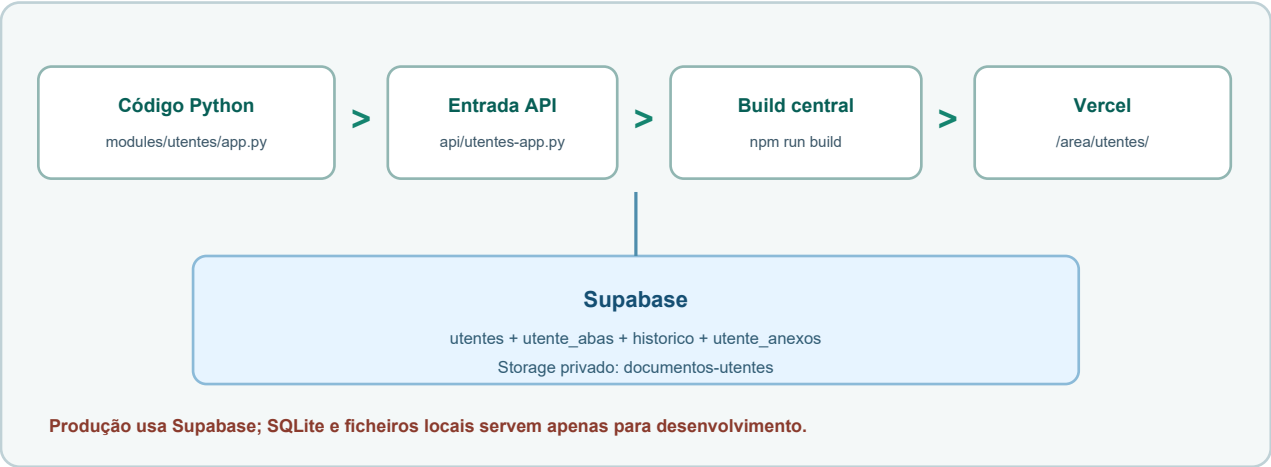
# Manual do Programador - Gestão de Utentes

Guia técnico da arquitetura atual, permissões, dados, edição manual, testes, Git e publicação da área de Utentes.

|                      |                                                                                                                                   |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>Ramo</b>          | Gestão de Utentes                                                                                                                 |
| <b>Destinatários</b> | Programadores e responsáveis técnicos                                                                                             |
| <b>Repositório</b>   | <a href="https://github.com/MenteMovimento/central-mente-movimento">https://github.com/MenteMovimento/central-mente-movimento</a> |
| <b>Site</b>          | <a href="https://central-mente-movimento.vercel.app">https://central-mente-movimento.vercel.app</a>                               |
| <b>Atualização</b>   | 23/07/2026                                                                                                                        |

# 1. Arquitetura atual de Utentes

O módulo é uma aplicação Python server-rendered integrada no build central. Em produção, dados e PDFs ficam no Supabase.



- A interface, as rotas HTTP, a validação e a persistência estão concentradas em app.py.
- A entrada serverless adapta a aplicação para a Vercel.
- A service role é usada apenas no servidor; nunca deve ser enviada ao browser.
- SQLite e anexos locais existem para desenvolvimento, não para persistência serverless.

# 2. Mapa de ficheiros

| Ficheiro                                   | Responsabilidade                                                                           |
|--------------------------------------------|--------------------------------------------------------------------------------------------|
| portal/modules/utentes/app.py              | Rotas, HTML/CSS/JS gerado, permissões, CRUD, tabs, pagamentos, anexos, impressão e backup. |
| portal/modules/utentes/api/index.py        | Adaptador/entrada do módulo quando executado isoladamente.                                 |
| api/utentes-app.py                         | Entrada serverless usada pelo projeto central na Vercel.                                   |
| portal/modules/utentes/supabase_schema.sql | Tabelas base, índices, RLS e bucket documentos-utentes.                                    |
| portal/modules/utentes/docs/               | PDFs PT/EN servidos no diálogo Manuais.                                                    |
| scripts/generate-manual-pdfs.py            | Fonte programática dos PDFs; altere aqui, não diretamente no PDF.                          |
| scripts/prepare-vercel-output.mjs          | Copia os artefactos necessários para public/ durante o build central.                      |

### 3. Modelo de dados

| Tabela/bucket      | Conteúdo                                            | Ligação                                        |
|--------------------|-----------------------------------------------------|------------------------------------------------|
| utentes            | Identificação base, contactos, estado e timestamps. | id é bigint identity.                          |
| utente_abas        | JSON/texto serializado de cada tab_key.             | Único por utente_id + tab_key.                 |
| utente_anexos      | Metadados do PDF, aba, tamanho e autor.             | FK para utentes; objeto no Storage.            |
| historico          | Ação, utilizador, alvo, detalhe e data.             | alvo_tipo='Utente' para histórico individual.  |
| documentos-utentes | PDFs privados.                                      | stored_name inclui o caminho lógico do utente. |

A tabela utente\_abas permite evoluir formulários extensos sem criar uma coluna para cada campo, mas exige normalização e validação cuidadosas no código Python.

### 4. Fluxo de leitura e escrita

- 1 A rota autentica a sessão e obtém o utilizador central.
- 2 can\_view\_utentes/can\_edit\_utentes e as variantes sensitive validam a ação.
- 3 A ficha base é lida de utentes; o separador é lido de utente\_abas.
- 4 O formulário é convertido para a estrutura esperada pela aba e validado.
- 5 save\_tab\_content faz upsert e log\_action cria a auditoria.
- 6 A resposta redireciona para o mesmo utente e tab com uma mensagem de resultado.

### 5. Permissões e classificação das abas

| Permissão              | Efeito                                                                       |
|------------------------|------------------------------------------------------------------------------|
| utentes.view           | Lista, ficha e abas normais: referência, emergência, proteção e pagamentos.  |
| utentes.edit           | Criação, edição e ações nas abas normais.                                    |
| utentes.view_sensitive | Consulta de inscrição, diagnóstica, atendimentos, plano individual e outros. |
| utentes.edit_sensitive | Alteração das cinco abas sensíveis.                                          |
| utentes.export         | Só mostra backup quando view_sensitive também está ativo.                    |
| utentes.delete         | Eliminação de utentes e dados associados.                                    |
| central.view_history   | Consulta do histórico global no menu.                                        |

UTENTES\_PUBLIC\_TABS é apenas uma classificação técnica de acesso normal. Não significa que os dados sejam públicos fora da aplicação.

## 6. Abas, JSON e compatibilidade

- TAB\_SECTIONS define ordem, chave e título. Ao adicionar uma aba, atualize classificação, renderização, parsing, impressão, backup e manual.
- normalize\_tab\_key rejeita chaves desconhecidas e regressa a uma aba válida.
- Campos antigos devem continuar a ser aceites quando a forma do JSON evolui.
- Pagamentos mantêm pag\_historico dentro do conteúdo da aba e têm funções próprias de normalização, edição e cancelamento.
- plano\_intervencao é guardado em utente\_abas como JSON com os gestores de caso, plano\_row\_count e campos plano\_{n}\_\*. Preserve as linhas e a respetiva ordem.
- O Plano Individual usa áreas de texto expansíveis e sheet-table; ao alterar a tabela, valide sempre a impressão para garantir que o conteúdo não fica cortado.
- O genograma é conteúdo estruturado; qualquer alteração deve preservar diagramas já guardados.
- Não renomeie tab\_key existentes sem uma migração explícita dos dados.

## 7. Anexos, backups e auditoria

- A aplicação aceita PDF e usa SUPABASE\_BUCKET, por defeito documentos-utentes.
- Os metadados são criados em utente\_anexos apenas depois do upload bem-sucedido.
- Downloads são servidos inline com no-store; o bucket permanece privado.
- Ao eliminar um utente, remova objetos, linhas de anexos, abas e ficha sem deixar órfãos.
- O backup cria ZIP em memória com indice.csv, ficha-completa.html, pagamentos.csv, historico.csv e anexos/.
- Não escreva conteúdo clínico, tokens ou service role em logs de erro.

## 8. Alterar código manualmente

- 1 Criar uma branch ou confirmar que o trabalho atual está identificado com git status.
- 2 Localizar o fluxo com rg antes de editar; app.py contém várias representações relacionadas do mesmo campo.
- 3 Alterar a fonte em portal/modules/utentes/app.py e, se necessário, o schema/migração.
- 4 Atualizar textos PT/EN, impressão, validação, permissões e histórico da mesma funcionalidade.
- 5 Rever git diff e procurar alterações acidentais, dados reais e segredos.
- 6 Executar testes locais e o build central antes de publicar.

```
git status --short
rg -n "texto-ou-função" portal/modules/utentes/app.py
git diff -- portal/modules/utentes/app.py
```

## 9. Testes mínimos por alteração

| Área alterada | Testes obrigatórios                                                              |
|---------------|----------------------------------------------------------------------------------|
| Lista/ações   | Pesquisar, abrir, editar, estado, apagar bloqueado/permitido e pagamento rápido. |
| Aba normal    | Ver/editar com permissões normais e confirmar histórico.                         |
| Aba sensível  | Testar conta sem sensitive, só view_sensitive e edit_sensitive.                  |
| Pagamentos    | Criar, editar, anular, cêntimos, Pago/Isento e resumo da lista.                  |
| PDF/anexos    | Upload, abertura, impressão, eliminação e utente errado bloqueado.               |

| Área alterada | Testes obrigatórios                                                       |
|---------------|---------------------------------------------------------------------------|
| Backup        | Permissão dupla, estrutura ZIP, PDFs e ausência de ficheiros temporários. |
| UI            | Desktop, tablet, tema claro/escuro, menus e clique fora para fechar.      |

## 10. Gerar os manuais

Os PDFs são artefactos gerados. A fonte editável é este script.

- Confirme os dois PDFs portugueses em `portal/modules/utentes/docs/`.
- Renderize todas as páginas e verifique cortes, sobreposição, tabelas e acentos.
- Mantenha o parâmetro de versão dos links quando publicar uma revisão.

```
python scripts/generate-manual-pdfs.py --only utentes
```

## 11. Build, Git e publicação

- 1 Executar `npm run build` na raiz do repositório e corrigir qualquer erro.
- 2 Confirmar que os PDFs e a rota de Utentes foram copiados para `public/`.
- 3 Testar localmente login, lista, ficha, permissões e PDFs.
- 4 Criar commit com mensagem objetiva e enviar para o repositório autorizado.
- 5 Publicar na Vercel e aguardar estado Ready.
- 6 Abrir produção, testar com uma conta de permissões limitadas e validar o PDF num novo separador.

```
npm run build
```

```
git status --short
```

```
git diff --check
```

```
npx vercel --prod --yes
```

## 12. Variáveis e segurança

| Variável                  | Regra                                                                     |
|---------------------------|---------------------------------------------------------------------------|
| SUPABASE_URL              | Pode identificar o projeto, mas deve ser configurada no ambiente correto. |
| SUPABASE_SERVICE_ROLE_KEY | Segredo de servidor. Nunca expor no HTML, JavaScript, Git ou screenshots. |
| SUPABASE_BUCKET           | Opcional; por defeito documentos-utentes.                                 |
| SQLite/local uploads      | Apenas desenvolvimento; não confiar para dados persistentes na Vercel.    |

## 13. Diagnóstico de falhas

| Sintoma          | Diagnóstico                                                                    |
|------------------|--------------------------------------------------------------------------------|
| 500 na rota      | Rever logs Vercel, import do handler, variáveis e exceção Supabase.            |
| Ficha não guarda | Verificar permissão, parsing do formulário, upsert e resposta da tabela.       |
| Anexo falha      | Verificar bucket, service role, MIME PDF, tamanho e metadados.                 |
| Aba desapareceu  | Verificar classificação sensitive e matriz da conta.                           |
| PDF antigo       | Confirmar hash do ficheiro publicado, query de versão e cache do browser/CDN.  |
| Função lenta     | Procurar pedidos repetidos, backups grandes e renderização de todos os anexos. |

## 14. Rollback e checklist final

- Preservar o deployment anterior e evitar alterações destrutivas de schema sem backup.
- Em falha de código, reverter o commit ou promover o deployment estável anterior.
- Em falha de dados, parar escritas, recolher evidência e aplicar migração corretiva testada.
- Confirmar CRUD, oito abas, permissões normais/sensíveis, pagamentos, anexos, impressão, indicadores, backup e histórico.
- Confirmar build sem erros, PDFs revistos, produção Ready e ausência de segredos/dados reais no Git.